중간고사 한 장 요약

범위 Lec 2~8 + 실습 · 각 장의 시험 포인트만 압축. 자세한 설명과 손계산은 각 lecture 페이지로

시험 정보 (교수님 메일 10/21 확인)

- **중간고사 10/29 (목) 18:00~21:00 (3시간) · 환경동101**. 일시는 교수님 메일(10/21), 장소는 TA 메일(10/22 23:30)로 확정. 형식(객관식/서술형, 오픈북 여부), 문항 수는 언급 없음 → PLMS 공지 확인
- 실습(lab) 내용에서도 시험 문제가 나올 수 있음: "Under consideration (Tentative)"라 확정 아님. 실습 문제를 못 풀면 그 실습 점수가 음수로 뒤집히는 부호 반전 감점 안이 함께 검토 중
- **결시하면 예외 없이 과목 낙제**: "If you miss any of the exams, you will fail this course"

1 Lecture 1 · 과목 소개

Abstraction	하위 수준의 구현 세부를 숨기고 interface만 제공. user는 "how"가 아니라 "what"에 관심 · 같은 프로세서도 engineer / architect / system programmer에게 다르게 보임
코드의 세 계층	High-level language (problem domain, portability) → Assembly (instruction의 text 표현) → Machine code (bit로 인코딩). 이 과목의 관심은 주로 assembly와 그 아래
topic 순서 (bottom-up)	bits/integers → floating point → machine-level (control, procedures, data) → optimization → memory hierarchy/cache → linking → exceptional control flow → virtual memory → dynamic memory allocation
대표 손계산	Xeon Phi (72 cores, core당 4 SMT threads, 512-bit vector register 32개): hardware thread $72 \times 4 = 288$개 · vector register 하나의 32-bit int 수 $512/32 = 16$개 · register 32개 총 크기 32×512 bit $= 2048$ B = 2 KB
machine code 해독	16-bit instruction, 앞 4 bit가 opcode, ADD(register) 형식은 `0001 ddd sss 000 ttt` : x18C4 = `0001 1000 1100 0100` → ADD R4, R3, R4

2 Lecture 2 · 비트 · 바이트 · 정수

$B2U_w(x) = \sum_{i=0}^{w-1} x_i\, 2^i$ · $B2T_w(x) = -x_{w-1}\, 2^{w-1} + \sum_{i=0}^{w-2} x_i\, 2^i$. two's complement는 MSB만 음의 weight (MSB = sign bit)

	식	w = 8	w = 16	w = 32
UMax	$2^w - 1$ (11⋯1)	255	65,535	4,294,967,295
TMax	$2^{w-1} - 1$ (01⋯1)	127	32,767	2,147,483,647
TMin	-2^{w-1} (10⋯0)	−128	−32,768	−2,147,483,648

비대칭	$\|TMin\| = TMax + 1$ (0이 양수 쪽 패턴을 씀) → **TMin의 절댓값은 표현 불가, −TMin = TMin**
C 타입 크기	(이 강의의 Linux x86-64 System V 환경) char 1 byte, short 2 bytes, int 4 bytes, long 8 bytes, pointer 8 bytes, float 4 bytes, double 8 bytes — 이 크기는 해당 환경의 ABI에 따른 것이며, ISO C가 모든 플랫폼에 동일하게 보장하는 값은 아니다
2의 거듭제곱	2^{10} K, 2^{20} M, 2^{30} G, 2^{40} T, 2^{50} P
Signed ↔ unsigned	비트 패턴은 그대로, 해석만 바뀜 (차이는 2^w). 한 식에 섞이면 signed가 unsigned로 변환. 비교에도 적용: **-1 < 0U는 거짓** (−1이 UMax가 됨)
Sign extension	부호 비트 복사 → 값 보존 (unsigned는 0 채움)
Truncation	상위 비트 버림 = mod 2^w → signed 값이 바뀔 수 있음 (53191 → −12345)
Negation	$-x = \sim x + 1$ · 예외는 TMin (~TMin + 1 = TMin)
Unsigned 덧셈 overflow	carry out 발생 (결과가 참 합보다 2^w 작음)
Signed overflow	같은 부호끼리 더했는데 결과 부호(MSB)가 바뀜 (PosOver/NegOver) · 부호가 다른 덧셈은 overflow 불가
Shift	x ≪ k = $x \cdot 2^k$ unsigned ≫는 logical(0 채움), signed ≫는 arithmetic(MSB 복제, −∞ 방향) → 음수 나눗셈은 bias: $(x + (1 \ll k) - 1) \gg k$ shift 양이 음수이거나 word size 이상이면 undefined

unsigned 함정	i ≥ 0은 항상 참 (무한 루프) · sizeof는 unsigned라 i − sizeof(int) ≥ 0도 항상 참
Endianness	2·4·8 byte 숫자에만 해당, 문자열(1 byte 단위)은 무관 · word의 주소 = 첫 byte 주소 · 64-bit 주소 공간 2^{64} B = 16 EiB

3 Lecture 3 · 부동소수점

$V = (-1)^s \times M \times 2^E$ · single: s 1 + exp 8 + frac 23 (bias 127) · double: s 1 + exp 11 + frac 52 (bias 1023) · bias = $2^{k-1} - 1$

경우	exp 필드	E	M
Normalized	00…0도 11…1도 아님	Exp − Bias	1.f (leading 1 생략)
Denormalized	00…0	1 − Bias	0.f
Special	11…1	f = 0 → ±∞ · f ≠ 0 → NaN	

denorm의 E = 1 − Bias인 이유	최소 normalized($E = 1 - Bias$)와 지수가 같아 간격이 유지됨 → 가장 큰 denorm에서 최소 normalized로 매끄럽게 연결 (gradual underflow)
Zero와 denorm	지수 필드와 fraction 필드가 모두 0이면 +0 또는 −0이며, 지수 필드가 0이고 fraction 필드가 0이 아니면 subnormal(denormalized) 값이다. +0은 모든 비트가 0이고 −0은 부호 비트가 1이다. +0과 −0은 비교상 같지만 1/(+0) = +∞, 1/(−0) = −∞로 구분된다
NaN 예	$\sqrt{-1}, \infty - \infty, 0 \times \infty$
15213.0	인코딩 = $1.1101101101101_2 \times 2^{13}$ → Exp = 140 → **0x466DB400** · 1.0 = 0x3F800000
성질	FP zero = integer zero (모든 비트 0) · s = 0이면 비트 패턴이 커질수록 값이 단조 증가 (s = 1이면 감소) → 부호, ±0, NaN을 처리하면 정수 비교로 대소 비교 가능

Rounding	기본은 round-to-nearest-even: 정확히 halfway일 때만 LSB가 짝수인 쪽 (2.5 → 2, 1.5 → 2). G/R/S (강의 표기: G = 결과에 남는 마지막 비트, R = 버려지는 첫 비트, S = R 뒤 나머지의 OR): R = 0 → 버림 · R = 1이고 S = 1 → 올림 · R = 1, S = 0 → halfway → G가 홀수면 올림 (짝수면 버림). round 뒤 $M \geq 2$가 되면 post-normalize (63 → 64)
곱셈·덧셈	곱셈·덧셈 모두 closed, commutative, monotonic(∞, NaN 제외) · **결합법칙·분배법칙 불성립** · float 반올림 예시는 (3.14f + 1e10f) − 1e10f == 0.0f이다. 접미사 f가 없는 (3.14 + 1e10) − 1e10은 double 연산이므로 0이 되지 않는다. 또한 1e20f × 1e20f − 1e20f × 1e20f는 float 곱셈에서 양의 무한대로 overflow한 뒤 ∞ − ∞를 계산하므로 NaN이 되지만, 대응하는 double 곱셈은 유한한 값이므로 두 곱이 같다면 차이는 0이다
C cast	double/float → int는 0 방향 truncate (범위 밖·NaN은 undefined) · int → double은 정확 · int → float은 frac 23비트라 rounding 가능 ($2^{24} + 1 = 16777217$ → halfway → 짝수 쪽으로 버려짐)
0.1	0.1처럼 분모가 2^k가 아닌 값은 이진에서 정확히 표현되지 않음 → 부동소수점은 정확한 결과를 표현할 수 없을 때 반올림 오차가 발생할 수 있지만, 모든 값과 연산에 반드시 오차가 생기는 것은 아니다

4 Lecture 4 · 머신 수준 기초

programmer-visible state	PC(%rip), register file, condition codes, memory (byte-addressable 배열) · R 접두 = 64-bit, E 접두 = 32-bit
aggregate type	Assembly에는 aggregate type(array, struct)이 없음: 연속된 byte일 뿐, 주소는 직접 계산
16개 64-bit register	%rax %rbx %rcx %rdx %rsi %rdi %rsp %rbp %r8~%r15 · %rsp = stack pointer (특수)
인자 순서	**%rdi → %rsi → %rdx → %rcx → %r8 → %r9**, 반환값 **%rax**

(R) = M[R] · D(R) = M[R + D] · D(Rb,Ri,S) = M[D + Rb + Ri × S]. S는 1, 2, 4, 8뿐, index register에 %rsp 불가

AT&T 문법	AT&T 문법은 source가 앞, destination이 뒤. `subq src, dst`는 dst − src

불가 조합	memory → memory (load 후 store로), destination이 immediate · suffix b/w/l/q = 1/2/4/8 byte, register 크기와 맞아야 함 (q ↔ %rax, l ↔ %eax)
leaq vs movq	**leaq는 주소만 계산 (memory 접근 없음, condition code 불변) · movq는 그 주소의 memory를 읽음**. lea로 x + x×2 = 3x 같은 산술도 한 명령어로
add/sub/left shift	add/sub/left shift는 signed·unsigned 구분 없음 (비트 동작이 같음) · right shift만 sarq(arithmetic)와 shrq(logical)로 나뉨 · 32-bit 목적지 명령은 상위 32 bit를 0으로 만듦

5 Lecture 5 · 제어

flag	뜻	해석
CF	MSB에서 carry out (뺄셈이면 borrow)	unsigned overflow / unsigned 비교
ZF	결과가 0	같음 비교
SF	결과의 MSB = 1	signed 음수
OF	two's complement overflow	signed overflow

cmpq·testq	`cmpq b, a` 는 **a − b**를, `testq b, a` 는 **a & b**를 계산해 flag만 세팅 (저장 안 함, 순서 주의) · testq는 CF, OF를 항상 0으로 · leaq, setX, movzbl은 flag를 바꾸지 않음
뺄셈의 CF = borrow	carry out이 **없을 때** CF = 1 (즉 unsigned로 a < b)
setX	조건을 dst의 최하위 1 byte에만 세팅 → movzbl %al, %eax로 마무리 (상위 32 bit도 0이 됨)

	조건	signed	unsigned
같음/다름	e: ZF · ne: ~ZF	공통	
부호	s: SF · ns: ~SF	공통	

	조건	signed	unsigned
크다/작다	g: ~(SF^OF)&~ZF · ge: ~(SF^OF) · l: SF^OF · le: (SF^OF)|ZF	▶ signed는 SF^OF	a: ~CF&~ZF · b: CF · be: CF|ZF

분기	분기는 C 조건의 **반대 조건**으로 else로 점프
분기가 비싼 이유	pipeline: 조건이 정해지기 전 다음 명령 위치를 모름
cmov	then과 else를 둘 다 계산해 하나를 고름 → **비싼 계산, 위험한 계산(null 역참조), 부작용 있는 식에는 사용 불가**
while 변환	-Og는 jump-to-middle (goto Test; Loop: body; Test: if(test) goto Loop) · -O1은 do-while conversion (guard + do-while) → 점프가 적음
switch	jump table은 case label 주소의 배열 (엔트리 8 byte, .quad), `jmp *.L4(,%rdi,8)` 로 한 번에 점프 · **경계 비교는 index = x − 최소 case로 정규화한 뒤 unsigned 비교 (ja) — 정규화된 index가 범위 밖이면 default로 감 (음수 입력도 case가 음수면 정규화 뒤 유효할 수 있음)** · 빠진 case는 default label, 같은 코드의 case는 같은 label을 가리킴 · sparse하면 if-else 연쇄로 컴파일

6 Lecture 6 · 프로시저

구분	register	규칙
caller-saved	%rax (반환값), %rdi~%r9 (인자 6개), %r10, %r11	caller가 call 전에 저장·후에 되돌림. callee는 마음껏 덮어써도 됨
callee-saved	%rbx, %rbp, %r12~%r15	callee가 쓰기 전에 저장·ret 전에 되돌림 (수정할 때만). %rsp는 특수한 형태
XMM	%xmm0~%xmm15	전부 caller-saved

call label	return address(다음 명령어 주소)를 push하고 점프
ret	pop해서 점프 → ret 때 %rsp가 return address를 가리켜야 함 · 되돌리기는 저장의 역순(LIFO)

인자·반환값	인자는 6개까지 register, 7번째부터 stack · 반환값 %rax
frame 구조 (낮은 주소 → 높은 주소)	Argument Build(optional), Local variables + Saved registers, Old %rbp(optional), Return addr., 7번째 이후 인자 (둘은 caller frame 쪽)
stack frame	호출마다 private storage (saved register, local 변수, saved return pointer)
stack discipline	P가 Q를 부르면 Q가 P보다 먼저 return (LIFO). mutual recursion도 같은 규약
재귀	호출마다 새 frame → 같은 %rbx를 써도 각자 push/pop으로 되돌림 · **종료가 아닌 호출 하나마다 16 byte (return address 8 + saved %rbx 8)** · x = $2^{64} - 1$이면 64 × 16 = 1024 B
pcount_r	수정하지 않는 경로(x == 0)에서는 callee-saved 저장을 건너뜀 · rep; ret은 ret과 같은 동작 (AMD용 최적화)

7 Lecture 7 · 데이터

1차원: 원소 i의 주소 = $N + \texttt{sizeof(T)} \cdot i$ · 2차원(row-major): 행 시작 = $N + i \cdot (C \cdot k)$ · 원소 = $N + i \cdot (C \cdot k) + j \cdot k = N + (i \cdot C + j) \cdot k$

배열·포인터	배열 이름은 첫 원소의 주소 · 포인터에 더하는 정수는 원소 크기만큼 곱해짐 (double *에 4를 더하면 32 B) · `val[i]` = `*(val+i)`
범위 검사 없음	val[5]의 값은 알 수 없음
Intel은 little endian	int 3은 03 00 00 00 · scale = 원소 크기 (int 4, long·pointer·double 8)
size_t	for (i = 0; i < 5; i++)는 size_t라 `cmpq $4, %rax` + `jbe` (unsigned 비교) · jump-to-middle이라 body 5번, 비교 6번
scale = 1·2·4·8	scale은 1, 2, 4, 8뿐 → 5×index는 `leaq (%rdi,%rdi,4)` (x + 4x), 12×idx는 `leaq (%rdi,%rdi,2)` + scale 4 (3 × 4)
nested 배열	주소 산술로 읽기 1번
multi-level (int *univ[3])	`Mem[Mem[univ+8·index]+4·dig]` : 읽기 2번, 행이 흩어져 있을 수 있고 행마다 길이가 달라도 됨
고정 크기 int[16][16]	행 크기 64 = 2^6이 상수라 `salq $6`

가변 a[n][n]	`imulq` 로 n·i. 주소는 둘 다 a + 4·n·i + 4·j

struct: field는 선언 순서대로, offset은 컴파일 시점에 결정 · 정렬: type이 B byte를 요구하면 주소는 B의 배수 · K = field 중 가장 큰 정렬 요구 → 시작 주소와 전체 길이 모두 K의 배수 (compiler가 internal + trailing padding 삽입)

struct S1	struct S1 {char c; int i[2]; double v;}: offset 0, 4, 16, sizeof 24 (padding 3 + 4)
순서를 바꾼 S2도 24 (끝 padding 7)	위치만 다름
공간 절약은 큰 type을 앞에	S4 {char c; int i; char d;} 12 B → S5 {int i; char c; char d;} 8 B · 절약이 생기는 경우는 작은 type 사이에 큰 type이 낄 때
struct 배열	a[idx]의 주소 = a + sizeof·idx (padding 포함 크기) · 끝 padding 덕분에 다음 원소도 정렬 유지
FP	인자 %xmm0, %xmm1, … 반환 %xmm0, 전부 caller-saved · 정수·포인터는 일반 register와 따로 셈 · XMM끼리는 movapd, memory ↔ XMM은 movsd · addss/addsd = scalar (낮은 칸 한 쌍), addps = SIMD (float 4개 동시)

8 Lecture 8 · 심화 주제

Memory layout	Text (0x400000 근처, read-only) → Data (global·static) → Heap (malloc, 위로 자람) → Shared Libraries → Stack (0x7FFFFFFFFFFF 근처, 기본 8 MiB, 아래로 자람)
buffer overflow	할당된 크기를 넘겨 쓰는 것: 기술적 원인 기준 보안 취약점 1위 · gets/strcpy/strcat/%s는 길이 제한이 없어 위험
echo (buf[4])	buf는 %rsp, return address는 %rsp+24 · 23바이트+null = 24는 buf+padding을 꽉 채움 · 24바이트+null이 return address 첫 byte를 0x00으로 덮음 · 25바이트 이상은 retq가 엉뚱한 주소로 점프 → Segmentation fault

방어	수준	내용	한계
fgets · strncpy · %ns	code	길이를 제한하는 함수로 교체	legacy code를 모두 고칠 수 없음

방어	수준	내용	한계
ASLR	system	시작 시 stack에 임의 공간 할당 → 주소 예측 불가	주소 유출 시 무력화 · 32비트는 엔트로피 낮음 · 고정 code 영역(PIE 미적용)은 무작위화 안 됨
Non-executable stack	system	실행 권한 명시, stack은 non-executable	ROP는 못 막음
Stack canary	compiler	buffer 뒤 special value, exit 전 check (%fs:0x28, __stack_chk_fail, -fstack-protector 기본)	off-by-one null 덮기는 통과 가능 · 앞쪽 local 변수 손상은 못 막음

ROP	ret(0xc3)으로 끝나는 gadget을 이어 붙여 기존 code를 재사용: non-executable stack과 ASLR(고정 code 영역) 우회 — PIE로 code까지 무작위화되면 주소 정보 없이는 어려움 · **canary는 일반적인 ROP의 추가 장벽** (return address를 덮는 과정에서 canary 값이 바뀌면 탐지됨, 단 canary 값 유출 시 우회 가능)
union	모든 member가 offset 0, size = 가장 큰 member, 한 번에 한 field만 (S1은 24 B지만 U1은 8 B) · union은 같은 bit의 재해석, cast는 값 변환. bit2float/float2bit에 사용
endian	dw.c = f0..f7일 때 little: s[0]=0xf1f0, i[0]=0xf3f2f1f0, l[0]=0xf7f6f5f4f3f2f1f0
big	그대로 0xf0f1, 0xf0f1f2f3, 0xf0f1f2f3f4f5f6f7

9 Data Lab에서 쓰는 개념

정수 문제	허용은 상수 0~255, ! ~ & ^ | + << >> (문제마다 부분집합과 최대 연산 수가 다름) · **금지: if·do·while·for·switch, macro, 함수 호출, &&, ||, −, 0xFF 초과 상수, int 외 타입: 위반하면 그 문제 0점**
부동소수점 문제	float를 비트 패턴을 담은 unsigned로 받음 · 모든 int/unsigned 연산, ||, &&, if, while, 임의 크기 상수 허용 · float 타입·연산·상수와 union/struct/array 금지
가정	32-bit two's complement, right shift는 arithmetic, word size 이상의 shift는 동작 예측 불가

도구	btest (결과 검사), dlc (규칙·연산 수 검사)
트릭	De Morgan으로 연산 치환 · !x는 0/1 (논리), ~x는 비트 반전 (~0 = −1) · 큰 상수는 shift로 조립 · 음수화와 뺄셈은 ~x + 1
sign mask	signed 값을 31칸 arithmetic right shift → 음수 아니면 0, 음수면 모든 비트 1 (−1) → AND·XOR·덧셈과 조합해 분기 없이 경우를 가름
IEEE 비트 조작	sign(31) · exp(30~23) · frac(22~0)으로 나눠 생각
먼저 exp로 경우를 나눔	0xFF (∞, NaN), 0 (0, denorm), 그 외 (normalized). denorm과 ∞/NaN을 normalized처럼 처리하면 틀림
덧셈 overflow 판정	같은 부호끼리 더했는데 결과 부호가 다르면 overflow · TMin 입력과 shift 양 0/31 경계를 항상 점검

10 객관식에서 제일 많이 틀리는 것

최종 점검: 함정 모음

-1 < 0U	거짓 · -1 < 0은 참 (섞이면 signed → unsigned)
cmpq %rsi, %rdi	%rdi − %rsi %rdi − %rsi (순서 반대 아님) · signed 대소는 SF^OF, SF 단독이 아님 · unsigned는 CF (jb = CF)
TMin	−TMin = TMin · TMin × 2 = 0 · TMax + 1 = TMin
int ↔ float cast	int → float는 값이 바뀔 수 있음 (16777217 → 16777216) · float → int는 0 방향 truncate
leaq	memory를 읽지 않음 · movq만 읽음 · memory → memory 불가
denorm	E = 1 − Bias (≠ −Bias), M = 0.f · +0 = −0으로 비교됨
switch 경계 비교 ja	index = x − 최소 case로 정규화 뒤 unsigned 비교 — 정규화된 index가 범위 밖일 때만 default (음수 입력은 case가 음수면 유효)
caller-saved	call 뒤에 caller-saved register 값이 남아 있다고 믿으면 틀림 · callee-saved를 썼으면 push/pop 짝 (역순)
5×	leaq (x + 4x) · scale은 1, 2, 4, 8뿐 · 포인터 +1은 원소 크기만큼
struct·union	struct는 선언 순서 그대로 · K = 가장 큰 정렬 요구, 전체 길이도 K의 배수 · union은 모두 offset 0